Geri besleme kısıtlı sıralı hesaplama: teori çerçevesi

Merkezi soru. *Can we learn hardware-realizable discrete state-transition systems without sacrificing the sequence-modeling capability of their continuous relaxations?*

Bugünkü merkez iddia (29 Ekim): Boolean recurrent modellerde durum kapasitesi, geçişlerin yerelliği ve geri besleme derinliği ayrı kısıtlardır. A5'te bunların bit maliyeti kesin olarak ayrıştırılabilir:

- genel tek seviyeli gerçekleştirmede 6 bit; bu, LUT6 ile bile sıkı;
- bağımsız dense bloklarla (kontrol yalnız harften, w = 2) K = 6'da 7 bit, K = 5'te 9 bit.

6 bitlik çözümün yapısı da anlaşıldı: 15 durumlu bir coset core ve iki koşullu toggle. Bu yapı blok core'larla genelleşiyor. Test edilen beş grubun dördünde (S_4, A_5, PSL(2,7), A_6) bilgi alt sınırı tek LUT6 seviyesinde, SAT'sız gerçekleşiyor. PSL(2,8)'de bu yapıyla alt sınır ancak fan-in 8'de geliyor. Fiziksel olarak fan-in 6, Fmax'te bir eşik: aynı state'te fan-in 11 → 6 ile Fmax 156 → 636 MHz. PSL(2,8)'de 12 bit / fan-in 6, 9 bit / fan-in 8'den hızlı (641'e karşı 317 MHz). Genel bir garanti de var (ikinci denetimden gelen yapıcı teorem): tek mertebeli bir üreteç seçilebiliyorsa her sonlu abelyen olmayan basit grup $S = \lceil \log_2|G| \rceil$ ve $K \le S + 2 - r_2(G)$ ile gerçekleşiyor. kb = 1 input-gated yapıda döngü, discretization hatasını çarparak büyütmez; dış hata terimleri yine birikebilir. Loghub'da hard modelin eski soft referansa göre açığı eğitimle 0,185'ten 0,02 bpb'ye indi (1,583 → 1,418; soft referans 1,398; farklı eğitimlerin karşılaştırması). Kalan fark, Boolean bitlerin sızdıran belleği temsil edememesiyle tutarlıdır; bu bir hipotez.

Bağımsız denetim (29 Ekim). Belge dışarıdan denetlendi ve denetimin bütün sayısal kontrolleri burada yeniden üretildi (`sw/fbseq/witness_check.py` → `witness_check.json`). Çıkan düzeltmeler ilgili bölümlere işlendi: §2, §3, §5.1, §5.3–5.5, §7, §8.

İkinci denetim (29 Ekim akşam). Bu denetim üç düzeltme ve bir teorem getirdi; hepsi burada ayrıca doğrulandı (`core_fiber.py --odd`, `theory_sat.py --coding`, `loop_bench.critical_path`):

- "Her grupta fan-in S + w − r_2" sanısı yanlış (Z_2, Z_8).
- Gauge, A5'te her etiketlemede bir core bitini düşüremiyor (sertifikalı karşı örnek).
- Tek mertebeli üreteçle bir kontrol bitini düşüren yapıcı teorem (§5.3).
- Fan-in sayımında ve SAT sorgusunda kontrol girişlerinin seçilebilirliği eksikti (Tanım 1 notu). FPGA ölçüm düzeneği ise bir LUT seviyeli döngülerde döngüyü değil kendi çıkış ağacını ölçüyordu (§5.4).

Bu belge, Faz 0 deneylerinden çıkan teorik çerçeveyi toplar. Beş ana bölümde ilerler:

1. Biçimsel model.
2. Bilinen sonuçların bize uyarlanması: zamanlama, grup yapıları, bilgi alt sınırı.
3. Kendi sonuçlarımız: A5 küçük vakasında kesin gerçekleştirme sınırları.
4. Öğrenilebilirlik ve kararlılık bulguları.
5. Literatürdeki yerimiz.

Literatür taraması Ekim 2026 sonunda üç hatta yapıldı. "Bulunamadı" ifadeleri bir arama sonucudur, yokluğun kanıtı değildir. Bütün sayılar `model/fbseq/*.json` dosyalarından gelir; bunları üreten kodlar `sw/fbseq/` altındadır.

---

0. Özet

	Sonuç	Durum
Zamanlama	Cascade'in iteration bound'u, bileşen döngülerinin maksimumuna eşittir. Bizim mimari sınıfında bu sınıra ulaşılabilir.	**Bilinen** (Leiserson–Saxe 1991, Thm 11); uyarlama
Grup yapıları	m üreteçli $G \leq S_n$'nin word problem'i tek LUT seviyesinde çözülebilir. Encoding'e göre fan-in ile state arasında bir ödünleşim var.	Parçaları bilinen; biçimsel ifadesi bulunamadı
Bilgi alt sınırı	Grup elemanını izlemek için $S \geq \lceil \log_2 \lvert G \rvert \rceil$	**Bilinen** (Myhill–Nerode)
A5, bağımsız dense bloklar	Kontrol yalnız harften ($w = 2$). Tek seviyede en az state: $K = 6$ için **7 bit**, $K = 5$ için **9 bit**; $K = 4$'te imkânsız	**Bizim** (alt grup taraması ile ispat)
A5, genel	$K = 6$ ile **6 bit**, yani alt sınır. Sınır sıkı.	**Bizim** (SAT tanığı + alt sınır; tanık 1 080 koşulla bağımsız denetlendi)
Coset core + fiber	$H \cong Z_2^k$ ile tek seviyede $S = \lceil \log_2 \lvert G \rvert \rceil$ (alt sınır), fan-in $\leq S - k + 1 + w$. Transversal $\mathbb{F}_2$ üzerinde lineer çözülünce A5, PSL(2,7), A6'da 300/300 etiketlemede fan-in $S + w - r_2$. Her etiketlemede değil (A5 karşı örneği); "her grupta" sanısı yanlış (Z_2, Z_8)	**Bizim** (yapı denetimde fark edildi; lemma ve ispat bizde; gauge sonucu gözlem)
Tek mertebeli üreteç teoremi	$G = \langle a, b \rangle$, $\lvert a \rvert$ tek: her etiketlemede $S = \lceil \log_2 \lvert G \rvert \rceil$, $K \leq S + 2 - k$. Guralnick–Kantor ile her sonlu abelyen olmayan basit grup için uygun alfabede	İkinci denetimden; ispat ve 81 etiketlemede kontrol bizde
Birleşik yapı (blok core + fiber)	S_4, A_5, PSL(2,7), A_6'da alt sınır $K \leq 6$ ile, tek LUT6 seviyesinde: 5, 6, 8, 9 bit. A_5'te $K = 5$ ile 7 bit (önce 9). PSL(2,8)'de $K = 6$'da 12 bit; alt sınır (9) ancak $K = 8$'de	**Bizim** (yapı ispatlı; minimum, alt sınıra eşit olduğu yerde kesin)
Fan-in'in fiziksel bedeli	Aynı state'te fan-in 6'yı aşınca Fmax 1,6–4,1 kat düşüyor, LUT sayısı 4–44 kat artıyor. $K \leq 6$'da her kodla tek LUT, 599–669 MHz. PSL(2,8)'de fiziksel optimum en az state değil, fan-in 6	**Bizim** ölçüm (düzeltilmiş düzenek)

	Sonuç	Durum
Fiziksel karşılık	Yalnız state döngüsü, iki seed ortalaması, düzeltilmiş düzenek (§5.4): 6 bit 643 MHz, 7 bit 614, 15 bit 688, öğrenilen 3-bit bloklar (2 seviye) 486. Tek seviyeli olanlar aynı hız ölçeğinde	**Bizim** ölçüm
Öğrenilen sınıfta temsil	Doğru A5 çözümü, eğitilen mimarinin içine yazıldığında %100 (yalnız head eğitildi)	**Bizim**
A5'in basit grup genellemesi	Yörünge argümanı: her basit grup için blok boyutuna göre bir alt sınır	Eleştirel okumadan; genelleme ve ispat bizde
Bilgi ve kayıp	L_min = H(Y | Z, W); fazlalık I(Y; H | Z, W)	**Bilinen** (standart)
İfade gücü	Boolean blok sıralaması: set/keep → star-free; + toggle → Z_2; 2 bit → Z_3; 3 bit → A5	Sürekli karşılığı bilinen (Sarrof 2024, Grazzi 2025); Boolean ölçümü bizim
Öğrenilebilirlik	Aynı parametrede hard − soft: soft eğitimden sonra +1,37, hard uyarlamadan sonra −0,23 bpb. Hard modelin eski soft referansa göre açığı 0,185'ten ~0,02'ye indi. A5'te softhard 2/4, hard-forward 0/8, öğrenilen codebook 0/32. Doğru dönüşümler donmuş ve identity kopyalıyken curriculum softhard'ı 0/4 → 3/4 çıkarıyor; mekanizma ve tablo keşfi ayrıştırılıyor (oracle seçim)	Ölçüm bizim; yöntemler literatürden
Kararlılık	Hata rekürsiyonu sınırı; $\mathbb{F}_2$ tam eşitliği; birleşen map'ler farkı siler, permütasyonlar durumları ayrı tutar; kb = 1 input-gated yapıda döngü hatayı çarparak büyütmez (dış terimler birikebilir; kb > 1'de Hamming uzaklığı büyüyebilir); sızdıran bellek hipotezi. Bellek–dayanıklılık bağı yalnız bağımsız 1-bit keep/toggle için geçerli; genel bloklar hata düzeltebilir (60 durum için ≥ 10 bit)	Matematiği standart; input-gated önermesi ve ölçümler bizim

1. Biçimsel model

Tanım 1 (K-LUT recurrent devre). Bir devrenin parçaları:

- State: s ∈ {0,1}^S, register'larda.
- Her adımda bir feedforward devre, girdiden ve alt katmanların state'inden bir **kontrol kelimesi** c_t üretir. Feedforward devre istenildiği kadar derin olabilir ve pipeline'lanır.
- Yeni state: s_{t+1} = F(s_t, c_t). F'nin her çıkış biti en fazla K girişli tek bir LUT'tur. LUT'un girdileri, bir D_j ⊆ {1..S} kümesindeki state bitleri ile w_j kontrol bitidir; |D_j| + w_j ≤ K.

Bu durumda döngüdeki mantık derinliği **1 LUT seviyesidir**. Genel durumda F, d seviyeli bir LUT ağıdır; döngü derinliği d olur.

Fan-in'in sayımı. Tanım 1'de her çıkış, kontrol bitlerini de seçebilir ($w_j \le w$).

- Bu belgedeki K değerleri, aksi belirtilmedikçe her çıkışın w kontrol bitinin hepsini okuduğu varsayımıyla hesaplandı: $|D_j| + w$.
- Kontrol bitleri seçilebilir sayılınca (`core_fiber.exact_fanin`, harf kodları a = 00, b = 01, e = 10), raporlanan 31 kodun hiçbirinin en büyük fan-in'i değişmedi.
- Ama gauge ve birleşik yapı aramaları ve ilk SAT sorgusu bu serbestliği kullanmıyordu. Bu yüzden bulamadıkları bir şey, genel modelde imkânsız demek değil.

Tanım 2 (block-local, dense). State bitleri bloklara ayrılır ve her bitin bağımlılık kümesi kendi bloğunun içinde kalır ($D_j \subseteq B(j)$). **Dense** ise $D_j = B(j)$, yani her bit bloğunun tamamını okur. Faz 0'daki B modelleri (kb = 1, 2, 3) dense block-local'dir.

Tanım 3 (gerçekleştirme). Bir $M = (Q, \Sigma, \delta, q_0)$ otomatının gerçekleştirmesi:

- injective bir encoding $e : Q \to \{0,1\}^S$;
- her $a \in \Sigma$ için $F(e(q), c(a)) = e(\delta(q, a))$ koşulunu sağlayan bir devre.

Maliyet dörtlüsü (S, D, L, R):

- S: state bit sayısı;
- D: döngü derinliği;
- L: LUT sayısı;
- R: routing (ölçümle).

$\mathscr{R}_K(M)$, fan-in K ile ulaşılabilir maliyet dörtlülerinin kümesidir; Pareto-optimal elemanları araştırıyoruz.

---

2. Zamanlama: Önerme A

A1 (ayrışma). Bileşenler arası bağlantı grafiği çevrimsizse bütün yönlü çevrimler bileşenlerin içindedir. Dolayısıyla global iteration bound

$$T_{IB} = \max_C \left(\sum_{v \in C} d(v)\right) / \left(\sum_{e \in C} w(e)\right)$$

bileşenlerin kendi bound'larının maksimumudur. *İspat:* Çevrimsiz bir grafikte bileşenler arası hiçbir kenar bir çevrimin üzerinde olamaz.

A2 (ulaşılabilirlik). Her düğümü tek LUT olan ve bileşenler arası yolları ileri yönlü olan sınıfımızda bu sınıra ulaşılabilir. Leiserson–Saxe'nin Theorem 11'i, unit-delay bir devrede $\Phi \le c$ yapan bir retiming'in var olmasını, $G - 1/c$ grafiğinde negatif ağırlıklı çevrim olmamasına bağlar. $c = 1$ için yeterli koşul, her çevrimde en az LUT sayısı kadar register bulunmasıdır.

Tek seviyeli F'de bu koşul eşitlikle sağlanır. Her state bitinin çıkışı tek bir LUT'tan doğrudan register'a gider; bu yüzden bit bağımlılık grafiğindeki her çevrim LUT ve register'dan dönüşümlü geçer ve #LUT = #register olur. Bir çevrim birden fazla LUT içerebilir; örneğin $s_0 \to s_1 \to s_0$ iki LUT ve iki register'dır. Bu durum sonucu değiştirmez.

Ideal retiming sonucunun dışında kalanlar ayrıca belirtilmeli: başlangıç durumu, sabit I/O zamanlaması, pipeline register'larının maliyeti ve fiziksel yönlendirme.

Tezde açıkça yazılması gerekenler:

1. Bileşenler arasına register eklemek retiming değil pipelining'dir. Latency artar, üst katman girişi aynı miktarda geciktirilir (skew).
2. Retiming başlangıç state'ini değiştirebilir (Touati–Brayton 1993).
3. Bağımsız dizilerde c-slow (Leiserson–Saxe Sec. 7), derin bir döngünün throughput'unu geri kazanır.
4. Mantıksal sınır ile post-route clock period aynı şey değildir. Routing gecikmesi ölçümle verilir (§5.3).

İki önemli ayrım (eleştirel bir okumadan, 28 Ekim):

- **Iteration bound bir alt sınırdır; her devrede ulaşılamaz.** Bölünemeyen düğümlerde fark açılabilir. Örnek: bir çevrimde birim gecikmeli üç düğüm ve iki register. Alt sınır 3/2, ama bir aralıkta iki düğüm bulunmak zorunda olduğundan en iyi değer 2. Bizim sınıfta her çevrimde LUT ve register sayısı eşit olduğundan (A2) alt sınıra ulaşılır.
- **Akış işleme hızı, üretim hızı değildir.** "Döngüde tek mux" iddiası girdinin baştan bilindiği durumlar içindir: puanlama ve sıkıştırmada encoder. Metin üretiminde ya da FPGA'de decode'da bir sonraki girdi head'in ve örneklemenin çıktısına bağlıdır; head de döngüye girer. Sıkıştırma case'inde encoder FPGA'de, decoder yerde olduğu için iddia encoder için geçerlidir.

Konum: Bilinen sonuçların bir corollary'si; atıflı bir lemma olarak sunulur.

---

3. Grup yapıları: Önerme B

m üreteçli $G \leq S_n$, artı identity harfi (m+1 harf, $w = \lceil \log_2(m+1) \rceil$ kontrol biti):

Encoding	Güncelleme	State	Bit başına fan-in	Yerellik
Positional, base ile: $e(p) = (p(\beta_1), \ldots, p(\beta_k))$	Sola çarpma, $p' = a \circ p$	$k \cdot \lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil + w$	Koordinat başına yerel: $(a \circ p)(\beta_i) = a(p(\beta_i))$
Aynı kod	Sağa çarpma, $p' = p \circ a$	–	–	**Yerel değil:** $(p \circ a)(\beta_i) = p(a(\beta_i))$ ve $a(\beta_i)$ base'de olmayabilir
Ters kod, base ile: $r = p^{-1}$, $e = (r(\beta_1), \ldots, r(\beta_k))$	Sağa çarpma	$k \cdot \lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil + w$	Yerel: $r' = a^{-1} \circ r$
Bütün n koordinat: $(p(1), \ldots, p(n))$	Sağa çarpma	$n \cdot \lceil \log_2 n \rceil$	$(m+1) + w$	Kopyalama: yeni koordinat i, eski a(i) koordinatından seçilir

Encoding	Güncelleme	State	Bit başına fan-in	Yerellik
One-hot (bit (i,j) = [g(i)=j])	İki yön	n^2	(m+1) + w	Her iki çarpma yönünde yerel

Sağa çarpmadaki karşı örnek, A5'te (0 tabanlı): p = (0,1,2,3,4), q = (0,3,2,4,1), a = (1,2,3,4,0). Eski koordinatlar eşit, p(2) = q(2) = 2; yenileri farklı, $(p \circ a)(2) = 3$ ve $(q \circ a)(2) = 4$. Kod injective olduğu için p geri çözülebilir, ama bu tek bir mux ile ve aynı fan-in'le yapılamaz.

Görevimiz sağa çarpma ($p_t = p_{t-1} \circ a_t$). §5.4'teki 15 bitlik elle kurulan devre zaten ters kodu, yani $r = p^{-1}$'in beş koordinatını saklıyor (`controls.py`).

Bütün sonlu basit gruplar 2 üreteçlidir: Steinberg 1962, Aschbacher–Guralnick 1984, Miller; CFSG'ye dayanır. Bu yüzden one-hot encoding ile fan-in 3 + 2 = 5 her basit grup için yeter.

Önemli uyarı: Bu sonuç yalnız her adımda alfabe {e, a, b} ise geçerlidir. Bir Krohn–Rhodes cascade'inde grup bileşeninin etkin girdisi (harf, alt state) çiftidir ve G'nin herhangi bir elemanı olabilir; bu durumda fan-in n ile büyür.

Konum: Parçaları bilinen (one-hot encoding, base); K-LUT diliyle yazılmış biçimsel bir ifadesi bulunamadı.

---

4. Bilgi alt sınırı

Önerme (Myhill–Nerode). Grup elemanını izleyen deterministik bir makine için $S \geq \lceil \log_2 |G| \rceil$.

Önerme (bilgi ve kayıp). Y bir sonraki byte, H geçmiş, Z = f(H) saklanan state, W head'in gördüğü pencere olsun. Sınırsız kapasiteli ve en iyi ayarlanmış bir head için en küçük kayıp L_min = H(Y | Z, W), bpb cinsinden. Geçmişin tamamını kullanan bir modele göre fazlalık:

$$H(Y \mid Z, W) - H(Y \mid H) = I(Y; H \mid Z, W)$$

Yani bellek, geçmişin tamamını değil, geleceği tahmin etmeye yarayan kısmını taşımak zorundadır. *İspat:* $E[-\log_2 q] = H(Y|Z,W) + E[KL(P \,\|\, q)] \geq H(Y|Z,W)$; fark ise koşullu karşılıklı bilginin tanımıdır.

Sonuç. d Boolean bit en fazla 2^d durum tutar. Sürekli bir relaxation aynı d koordinatta çok daha fazla ayırt edilebilir değer kullanabilir. Bu yüzden soft modelin başarısı, aynı d ile hard bir gerçekleştirmenin var olduğunu göstermez.

İspat: İki girdi dizisi farklı grup elemanlarına ($g \neq h$) götürüyorsa, g^{-1}'e götüren bir sonek ilkini identity'ye götürür, ikincisini götürmez. Bu yüzden iki dizi aynı state'e düşemez; state sayısı en az |G| olmalıdır. A5 için: **6 bit.**

---

5. Gerçekleştirilebilirlik: A5 küçük vakası (bizim sonuçlarımız)

Görev: $p_t = p_{t-1} \circ a_t$, $a_t \in \{(01234), (012), e\}$, w = 2. Bu, `fbseq.py`'deki A5 görevi.

5.1 Block-local yapı ve alt gruplar

Önerme C1. Dense block-local bir gerçekleştirmede her blok, makinenin substitution-property (SP) partition'ına karşılık gelen bir bölüm makinesidir. Sağdan çarpan grup makinesinde SP partition'lar tam olarak alt grupların sağ coset'leridir (H p). Bütün state'in injective olması $\bigcap_i H_i = \{1\}$ koşuluna denktir.

İspat:

1. **SP partition → alt grup.** Bir blok yalnız kendi bitlerine baktığı için, bloğun ayırt ettiği state'ler partition'ı SP özelliğini sağlar: $p \equiv q \Rightarrow p \circ a \equiv q \circ a$. Σ, G'yi ürettiğinden bu her $g \in G$ için $p \circ g \equiv q \circ g$ demektir. H = [1] sınıfı çarpmaya göre kapalıdır, yani bir alt gruptur. $p \equiv q \Leftrightarrow p\,q^{-1} \equiv 1 \Leftrightarrow Hp = Hq$.
2. **Injectivity.** p ile q bütün bloklarda aynı $\Leftrightarrow p\,q^{-1} \in \bigcap_i H_i$.

SP partition kavramı Hartmanis–Stearns'e (1961–1966) aittir; grup makinelerindeki karşılığı klasiktir. Bizim katkımız, bunun LUT fan-in kısıtıyla birleştirilmesi.

Sonuç. Sabit tutulanlar:

- alfabe kodlaması c : Σ → {0,1}^w;
- her bloğun kontrolü yalnız o adımdaki harften gelir; alt katman ya da başka blokların state'i kontrole girmez;
- bloklar birbirinden bağımsız ve dense.

Bu sınıfta en küçük state:

$\beta_{K,w}(G; \Sigma, c) = \min \Sigma_i \lceil \log_2[G : H_i] \rceil$, koşullar $\bigcap H_i = \{1\}$ ve $\lceil \log_2[G : H_i] \rceil + w \le K$.

Bu varsayımlar gerekli. Tanım 1 daha geniştir: kontrol alt katmanın state'inden üretilebilir ve w_j bitten bite değişebilir. Bir blok başka blokların state'ine bağlı kontrol alırsa, tek başına grup makinesinin bir SP partition'ı olmak zorunda değildir. §5.3'teki 6 bitlik tanık tam olarak bu yüzden sınıfın dışında kalıyor. Aşağıda $\beta_{K,2}$, kısaca β_K yazılıyor.

5.2 A5 için değerler

A5'in 59 alt grubu tarandı (`theory_sat.subgroups`). Mertebe dağılımı 1, 2 (15), 3 (10), 4 (5), 5 (6), 6 (10), 10 (6), 12 (5), 60.

- **K = 4:** Blok en fazla 2 bit olabilir, yani indeks ≤ 4. A5'in indeksi ≤ 4 olan öz alt grubu yok. **İmkânsız.**
- **K = 5:** Blok en fazla 3 bit, yani indeks ≤ 8. Adaylar yalnızca A4 (indeks 5) ve D10 (indeks 6).
- İki farklı A4'ün kesişimi Z_3'tür (iki nokta sabit).
- $A4 \cap D10 = Z_2$: D10 beş nokta üzerinde transitive, bu yüzden bir noktanın stabilizer'ı mertebe 2.
- İki D10'un kesişimi her zaman tek bir involution: 15 involution'ın her biri tam iki D10'da bulunuyor.

İki blok yetmiyor, üç blok yetiyor. $\boldsymbol{\beta_5 = 9}$ (örnek: üç D10).

- **K = 6:** Blok en fazla 4 bit, yani indeks ≤ 16. $A4 \cap V4 = \{1\}$ olan bir çift var; örneğin A4 ile mertebeleri aralarında asal olan Z5 de aynı işi görür. $\boldsymbol{\beta_6 = 7 = 3 + 4}$**.** Daha azı mümkün değil:

3+3 bileşimleri yukarıdaki kesişimler yüzünden tutmuyor; indeksi ≤ 4 olan alt grup olmadığından 2+4 de yok.

Doğrulama: Üç çözüm de rastgele dizilerde %100 doğru (`theory_sat.check_family`: 500 dizi × 512 adım).

Yörünge argümanı ve basit gruplar için genelleme (eleştirel okumadan; tarama ispatını genelleştirir). G basit, blok b bitlik olsun ve yalnız kendi state'ini ve girdiyi görsün. G'nin bloğun başlangıç durumunun yörüngesi üzerindeki etkisi ya triviyaldir ya da sadıktır, çünkü çekirdek G'nin bir normal alt grubudur. Yörünge boyutu bir alt grubun indeksidir; dolayısıyla |G|'yi böler ve triviyal değilse G'nin en küçük öz alt grup indeksinden μ(G) küçük olamaz.

Bundan bir alt sınır çıkar. Blokların yörünge boyutları $o_1, \ldots, o_r$, her biri 2^{b_i}'den küçük ya da eşit, |G|'yi bölen ve 1 ya da ≥ μ(G) olan sayılardır. State'in G'nin elemanlarını ayırabilmesi için $\prod o_i \geq |G|$ gerekir. Bu, en küçük $\sum b_i$ için genel bir alt sınırdır.

A5 için (μ = 5):

- 2 bitlik bloklar triviyaldir.
- 3 bitlik bloklarda yörünge 5 ya da 6 olabilir. İki blok en fazla $6 \cdot 6 = 36 < 60$ ayırt eder, yani en az 9 bit gerekir.
- 4 bitlik bloklarda yörünge en fazla 15 olabilir. 4 + 2 bit yetmez, 3 + 3 bit 36 verir; yani en az 7 bit gerekir.

Bu, §5.2'deki taramayı doğrular ve aynı akıl yürütme her basit grup için çalışır.

5.3 Block-local olmayan: sınır sıkı

SAT/SMT sorgusu (`theory_sat.py`, z3, 6 344 s): S = 6 ve K = 6 için, her bitin en fazla 4 state bitine ve 2 kontrol bitine bağlı olduğu injective bir encoding **var**. Bağımsız doğrulama (`witness_check.py`, z3 kullanmıyor):

- 60 kodun hepsi farklı;
- 60 durum × 3 harf × 6 bit = 1 080 koşulun hepsi sağlanıyor;
- rastgele dizilerde %100 doğru (300 dizi × 512).

Tam geçiş denetimi, rastgele testten güçlüdür. Geçerli 60 koddan birinde başlayan devre her harfte doğru geçiş yapıyorsa, tümevarımla her sonlu kelimede doğrudur. Kapsam dışında kalanlar: kullanılmayan 4 koddan başlamak ve fiziksel bir hatadan sonra toparlanmak.

Alt sınır da 6 olduğundan **A5 için tek seviyeli, fan-in 6 gerçekleştirmenin en küçük state'i tam olarak 6 bittir.** Bağımsız dense blok kısıtı bu değeri 7'ye çıkarıyor.

Bir notu açıkça yazmak gerekir: genel bir 6 bitlik kodda bir sonraki state biti 6 state bitine ve kontrol bitlerine bağlı olabilir. Bu da tek bir LUT6'ya sığmayabilir. SAT tanığı tam bu engeli aşan bir encoding buluyor: her bit yalnız 4 state bitine ve 2 kontrol bitine bağlı. Yani 6 bitlik minimum **fan-in 6 ve tek seviye kısıtı altında da** geçerli. Bağımsız blok kısıtının A5'teki bedeli tam 1 bit.

Tanığın yapısı: coset core + fiber (`witness_check.json`; yapıyı denetim fark etti). Bitler 0'dan numaralanıyor.

- **Core:** (s_0, s_1, s_2, s_5) kapalı bir 4 bitlik çekirdek; bu dört bitin sonraki değeri yalnız kendilerine ve harfe bağlı. Core'un 15 erişilebilir değeri var; her biri V4p biçiminde bir sağ coset. Burada V4, 2. noktayı sabitleyen mertebe 4'lük alt grup: {e, (01)(34), (03)(14), (04)(13)}.
- **Fiber:** s_3 ve s_4, coset'in içindeki 4 elemanı ayırıyor. Her biri bir toggle: $s'_j = s_j \oplus f_j$(core, harf). s_3'ün flip'i (s_0, s_2, s_5)'e, s_4'ünki (s_0, s_1, s_2)'ye bakıyor. Fan-in: 1 kendi biti + 3 core biti + 2 harf biti = 6.

Sezgi: Bu bir Krohn–Rhodes cascade'i, ama tek LUT seviyesinde. Alt bileşen (fiber), üst bileşenin (core) **eski** değerini okuyor. Senkron bir adımda bu değer zaten register'da hazır olduğundan döngü derinliği artmıyor. Fiber blokları kendi başına SP partition olmadığı için §5.1'deki alt sınır bu yapıya uygulanmıyor.

Lemma (coset core + Z_2^k fiber). $H \le G$, $H \cong Z_2^k$ olsun ve $c = \lceil \log_2[G:H] \rceil$ yazalım. Sağa çarpan G makinesinin tek LUT seviyeli, S = c + k bitlik bir gerçekleştirmesi vardır. Fan-in, core bitleri için en fazla c + w, fiber bitleri için en fazla 1 + c + w'dir.

İspat. Sağ coset'ler için bir T transversal'i seçilir; her p tek biçimde $p = h \cdot t$ yazılır ($h \in H$, $t \in T$). Bir harf için $p \circ a = h \cdot (t \circ a)$ olur. $t \circ a$, Ht' coset'indedir; yani $t \circ a = \gamma(t, a) \cdot t'$, $\gamma \in H$.

- Core, coset'i ($Ht \mapsto Ht'$) yalnız kendi değeri ve harfle günceller. Bu, §5.1'deki SP partition'dır.
- Fiber $h \mapsto h \cdot \gamma(t, a)$ olur. $H \cong \mathbb{F}_2^k$ koordinatlarında bu, her koordinatın kendi eski değeri ile γ'nın ilgili koordinatının XOR'udur.

Her şey eski state ve harften hesaplandığı için döngü tek seviyedir. (core, fiber) ikilisi p'yi belirlediği için kod injective'dir. ∎

Önerme (bu biçimdeki her kod lemmadan gelir). Core'u H'nin coset'lerini kodlayan ve fiber bitleri (core, harf)'e koşullu toggle olan her injective kod, lemmanın inşasıyla elde edilir. Serbestlikler: coset etiketlemesi, transversal ve H'nin $\mathbb{F}_2^k$ tabanı.

Taslak. Bir coset'e dönen kelimeler fiber'da $t^{-1}Ht$ ile çarpma etkisi yapar. Toggle koşulu bu etkinin XOR'la bir sabit olmasını gerektirir. Bu yüzden coset içindeki fiber kodu, H'nin $\mathbb{F}_2^k$'ye bir izomorfizmi artı bir ötelemedir. Komşu coset'ler arasında tek bitlik toggle'lar bu izomorfizmin değişmesine izin vermez; core grafiği bağlantılı olduğundan izomorfizm her yerde aynıdır. Ötelemeler transversal seçimine karşılık gelir.

Gözlem: state her zaman alt sınırda. $[G:H] = |G| / 2^k$ olduğundan $c + k = \lceil \log_2|G| - k \rceil + k = \lceil \log_2|G| \rceil$. Yani lemma, Z_2^k alt grubu olan her grupta Myhill–Nerode sınırına ulaşıyor. $H = \{1\}$ alındığında (k = 0) bu, tek dense blok demek: fan-in S + w. Lemmanın kazancı fan-in'de: en büyük k, yani grubun 2-rank'i $r_2(G)$ seçilirse fan-in S + w'dan $S - r_2 + 1 + w$'ya iner.

Gauge: transversal seçimi lineer bir problem. t_k yerine $h_k \cdot t_k$ alınırsa, k coset'indeki fiber kodu $\delta(h_k)$ kadar kayar. Bir fiber fonksiyoneli ℓ için flip şu olur:

$f(k, a) = \ell(\gamma(k, a)) \oplus \beta(k) \oplus \beta(k')$, $\beta(k) \in \mathbb{F}_2$ serbest.

"Flip yalnız D core bitlerine bakıyor" koşulu $f(k, a) = g(\text{kod}_D(k), a)$ demektir; bu (β, g) bilinmeyenlerinde $\mathbb{F}_2$ üzerinde lineer bir denklem sistemi. Etiketleme sabitken sistem Gauss eliminasyonuyla kesin çözülür. Çözümden transversal ve taban okunur; kod sonra kaba kuvvetle yeniden doğrulanır (`core_fiber.py`).

A5'te sonuç. H = V4 ile c = 4, k = 2, yani S = 6 = alt sınır.

- Lemma bu 6 biti her etiketlemede **K = 7** ile veriyor.
- K = 6 için her fiber flip'inin 4 core bitinden yalnız 3'üne bağlı olması gerekiyor.
- **Rastgele transversal ile:** 10 000 denemenin hiçbiri K = 6'ya inmedi. Ailede simulated annealing de (8 × 20 000 adım) bulamadı.
- **Gauge çözülünce:** rastgele 300 coset etiketlemesinin **300'ü** K = 6 veriyor. SAT'ın 6 344 saniyede bulduğu sonuca milisaniyeler içinde ulaşılıyor. SAT tanığının kendisi de bu ailenin üyesi: aile onu birebir üretiyor.
- Önceki sürümdeki "K = 6 yalnız özel etiketlemelerle" ifadesi bu yüzden yanlıştı. Nadir olan etiketleme değil, doğru transversal; o da lineer olarak bulunuyor.

Başka gruplar (`core_fiber.json`; harfler iki üreteç ve identity, w = 2; Z8'de tek üreteç, w = 1):

Grup	|G|	S = ⌈log$_2$|G|⌉	r$_2$	Dense fan-in	Lemma	Rastgele transversal	Gauge çözülmüş (300 etiketleme)
Z_2^3	8	3	3	5	3	3	3 (300/300)
Z_8	8	3	1	4	4	4	4 (300/300)
S_4	24	5	2	7	6	5 (9/300)	**4** (5/300), 5 (277/300)
A_5	60	6	2	8	7	7	**6** (300/300)
PSL(2,7)	168	8	2	10	9	9	**8** (300/300)
A_6	360	9	2	11	10	10	**9** (300/300)

Okuma:

- **Kontrol grupları beklendiği gibi.** Z_2^3'te her bit kendi kendini toggle'lıyor (fan-in 1 + w). Z_8'de 2-rank 1 ve kazanç yok: ikili sayacın en üst biti bütün alt bitlere bakmak zorunda.
- **Basit gruplarda (A_5, PSL(2,7), A_6)** gauge çözülünce fan-in **S + w − r$_2$**'ye iniyor, yani core'un kendi fan-in'ine. Fiber artık darboğaz değil.
- **S_4'te bir adım daha aşağı (4).** Core'un 6 durumu 3 bitin 8 koduna seyrek yerleşiyor. Etiketlemelerin ~%2'sinde her core biti yalnız 2 core bitine bakıyor; bu dört alt grubun hepsinde oluyor, normal V4'e özgü değil. Basit gruplarda 300 etiketlemenin hiçbirinde toplam fan-in c + w'nın altına inmedi. Core etiketlemesi ayrıca aranmadı.

Durum. Lemma ve "her kod lemmadan gelir" önermesi ispatlı. Gauge sonucu bir gözlem, ispat yok. Üç basit grupta sistem 900 etiketlemenin hepsinde çözüldü; S_4'te 300'ün 282'sinde çözüldü.

Düzeltme 1: "her grupta S + w − r$_2$" sanısı yanlış (ikinci denetim; burada yeniden doğrulandı). Önceki bir sürüm, minimum state'te fan-in S + w − r$_2$(G)'nin her grupta ulaşılabilir olduğunu sanı olarak yazıyordu. Bu yanlış; kendi tablomuzdaki Z_2^3 satırı (formül 2, bulunan 3) zaten buna işaret ediyordu.

- **Z_2, harfler {e, toggle}:** S = 1, w = 1, r$_2$ = 1, formül K = 1. Ama s′ = s ⊕ u iki girişe bakmak zorunda: K = 2.
- **Z_8, harfler {e, +1}:** formül K = 3. Identity'nin kodu 000'a sabitlenince 7! = 5 040 minimum-bit kodlama kalıyor; hepsinde en büyük fan-in 4. Kontrol bitleri de seçilebilir sayıldı; burada

ayrıca tarandı.
- Denetimde hesaplamadan bağımsız bir ispat da var: K ≤ 3 olsaydı üretecin güncellemesi $\mathbb{F}_2^3$ üzerinde affine bir permütasyon olurdu, ve affine bir permütasyon mertebe 8 olamaz.

Düzeltme 2: gauge her etiketlemede bir core bitini düşüremez (A5, sertifikalı karşı örnek; burada `gauge()` ile yeniden doğrulandı).

- Etiketleme: V4 = {e, (0 2)(3 4), (0 3)(2 4), (0 4)(2 3)}, coset etiketleri [8, 12, 5, 7, 15, 1, 6, 13, 2, 9, 10, 0, 11, 3, 14].
- Dört core bitinden hangisi çıkarılırsa çıkarılsın, yalnız (1,1) fonksiyoneli için sistem çözülüyor; (1,0) ve (0,1) için çözülmüyor. Yani uygulanabilir fonksiyoneller tek boyutlu bir uzayda kalıyor ve iki bağımsız fiber koordinatı seçilemiyor. Gauge burada K = 7'de kalıyor.
- Denetimin rank karakterizasyonu: core bitleri D için uygulanabilir fonksiyoneller bir alt uzay W_D oluşturur. Fiber'in her koordinatı en fazla d core biti okuyabiliyorsa, gerek ve yeter koşul dim span(∪_{|D| ≤ d} W_D) = k.

Teorem (tek mertebeli üreteç; ikinci denetimden, ispatı burada kontrol edildi). Varsayımlar:

- G = ⟨a, b⟩ ve a'nın mertebesi m tek;
- H ≤ G, H ≅ Z_2^k;
- harfler {a, b, e}, kodları a = 00, b = 01, e = 10;
- her çıkış yalnız ihtiyaç duyduğu kontrol bitlerini okuyabilir.

O zaman **her** core etiketlemesi için tek LUT seviyeli bir gerçekleştirme vardır: S = c + k = $\lceil \log_2|G| \rceil$ ve K ≤ c + 2 = S + 2 − k.

İspat.

1. a'nın sağ coset'ler üzerindeki her yörüngesi tam m uzunluğundadır. Ht·a^j = Ht olsaydı t a^j t^{-1} ∈ H olurdu; bu elemanın mertebesi hem tek hem 2'nin kuvveti, yani identity. Dolayısıyla a^j = e.
2. Her yörüngede temsilciler t_0, t_0a, …, t_0a^{m−1} seçilir. Transversal sağdan a ile çarpmaya kapalı olur: γ(x, a) = e, ayrıca γ(x, e) = e.
3. Fiber yalnız b harfinde değişir. u = 1 yalnız b için olduğundan z′_j = z_j ⊕ (u ∧ Γ_j(x, b)). Bu çıkış c core bitine, kendi bitine ve tek bir kontrol bitine bakar: c + 2.
4. Core çıkışları c state biti ve 2 kontrol bitine bakar: c + 2. ∎

Fark şurada: gauge bir **core bitini** düşürmeye çalışıyor; bu inşa bir **kontrol bitini** düşürüyor. Bu yüzden Düzeltme 2'deki etiketlemede de K = 6 veriyor.

Doğrulama (`core_fiber.py --odd` → `odd_generator.json`; fan-in, kontrol bitleri seçilebilir sayılarak, tam ölçüldü):

Grup	a'nın mertebesi	S	k	Sınır c + 2	Ölçülen K (her etiketlemede)
A_5	5	6	2	6	6 (21 etiketleme, karşı örnek dahil)
PSL(2,7)	7	8	2	8	8 (20)
A_6	5	9	2	9	9 (20)
PSL(2,8)	7	9	3	8	8 (20)

Bütün durumlarda fiber bitleri yalnız u kontrol bitini okuyor.

Sonuç (bütün sonlu abelyen olmayan basit gruplar). Böyle bir grup 2-grubu olamaz, bu yüzden tek asal mertebeli bir a elemanı vardır. Guralnick–Kantor (2000, J. Algebra 234, 743–792) her identity dışı elemanın bir üreteç çiftine tamamlanabildiğini gösteriyor; buradan G = ⟨a, b⟩ olan bir b bulunur. H olarak en büyük elementary abelian 2-alt grup alınır (k = $r_2(G)$). Sonuç: S = ⌈$\log_2|G|$⌉ ve K ≤ S + 2 − $r_2(G)$.

Kapsam:

- Bu, **uygun seçilmiş** bir {e, a, b} alfabesi için bir varlık sonucu. Verilen keyfi bir üreteç çiftini kapsamıyor; alfabeyi değiştirmek bir kelime görevinde bedava değil.
- Fan-in için bir üst sınır; optimumluk iddiası yok. Birleşik yapı test edilen gruplarda daha iyi (aşağıda).
- Grup teorisi kısmı klasik. LUT gerçekleştirmesine bağlanmasının literatürde ilk olduğunu iddia etmiyoruz.

Birleşik yapı: blok core + fiber (`realize.py` → `realize.json`). İki yapı tek bir şemada birleşiyor.

- **Core:** H ≅ Z_2^k'yi içeren O_1, …, O_r alt grupları seçilir; koşul O_1 ∩ … ∩ O_r = H. Blok i, O_i p sağ coset'ini b_i = ⌈$\log_2$[G:O_i]⌉ bitte tutar ve yalnız kendisine ve harfe bakar (fan-in b_i + w).
- **Fiber:** H için k toggle biti. Gauge §5.3'teki gibi lineer çözülür; flip'in okuduğu core bitleri D, fan-in 1 + |D| + w.
- **Toplam:** S = Σb_i + k.

Doğruluk. ∩ O_i p = (∩ O_i) p = H p olduğu için bloklar birlikte H-coset'ini belirler. Gerisi lemmanın ispatıyla aynı; döngü tek seviye. ∎

Özel durumlar: H = 1 ise bağımsız dense bloklar (β_{K,w}, §5.1). r = 1 ve O_1 = H ise tek dense core + fiber.

Arama ve doğrulama:

- Her elementary abelian 2-alt grup sınıfından bir temsilci alındı.
- En küçük Σb_i'yi veren aileler (+1 bit pay, ilk 12 aile) ve aile başına 40 blok etiketlemesi denendi.
- Bulunan her kod iki yoldan doğrulandı: her bitin bağımlılık kümesi kaba kuvvetle, ve `loop_bench`'in ürettiği Verilog tabloları 3 000 rastgele harfle simülasyonla.

Bulunan en küçük state (üst sınırlar; harfler iki üreteç ve identity, w = 2):

Grup	Alt sınır	K = 4	K = 5	K = 6	K = 7	K = 8	Önceki en iyi
S_4	5	**5**	**5**	**5**	**5**	**5**	β_4 = 5
A_5	6	–	7	**6**	**6**	**6**	K = 5'te β = 9
PSL(2,7)	8	–	**8**	**8**	**8**	**8**	K = 6'da β = 8; K = 5'te β = 9
A_6	9	–	12	**9**	**9**	**9**	K = 6'da β = 10; 9 bit için K = 9 (gauge)

Grup	Alt sınır	K = 4	K = 5	K = 6	K = 7	K = 8	Önceki en iyi
PSL(2,8)	9	–	–	12	10	**9**	–

Kalın değerler alt sınıra eşit, yani o K için kesin minimum. "–", yapının bu K'da bir şey bulmadığı anlamına geliyor; imkânsızlık kanıtı değil. Örneğin A5'te indeksi ≤ 4 olan öz alt grup yok, bu yüzden K = 4'te blok kurulamıyor.

Öne çıkan yapılar:

- **A_5, K = 5, 7 bit:** A4 bloğu (indeks 5, 3 bit) ve D10 bloğu (indeks 6, 3 bit); kesişimleri Z_2, yani 1 fiber biti. Bağımsız bloklarla K = 5'te 9 bit gerekiyordu. 6 bitin K = 5'te mümkün olup olmadığı açık. Kontrol girişlerini seçilebilir sayan SAT (`theory_sat.py --coding`, `sat_a5_ctrl_{a,b,e}.json`) 2 bitlik harf kodlamasının üç sınıfında da (ortadaki kod 00'a a, b ya da e) 6 saatte (21 600 s) **unknown** ile bitti; iki kontrol bitini zorunlu sayan kısıtlı sorgu da 4 saatte unknown (`sat_a5.json`). Yani ne bir tanık ne bir imkânsızlık kanıtı var.
- **PSL(2,7), K = 5, 8 bit = alt sınır:** iki S4 bloğu (indeks 7, 3'er bit), kesişim V4, 2 fiber biti. Her fiber biti en fazla 2 core bitine bakıyor.
- **A_6, K = 6, 9 bit = alt sınır:** iki S4 bloğu (indeks 15, 4'er bit), kesişim Z_2, 1 fiber biti, en fazla 3 core bitine bakıyor.
- **PSL(2,8), alt sınıra K = 6'da ulaşılamadı.** Bu yapıyla K = 6'da 12 bit (positional), K = 7'de 10 bit, 9 bit ancak K = 8'de geliyor. 9 bit için Z_2^3'ün 63 coset'i 6 bitlik dense bir core'da tutuluyor. Sebep yapısal: PSL(2,8)'de indeksi ≤ 16 olan tek öz alt gruplar 9 nokta stabilizer'ı (2^3:7, indeks 9). Bunların kesişimi ya Z_7 ya trivial; hiçbiri Z_2^k değil. Bu yüzden K = 6'da ailede yalnız bağımsız bloklar kalıyor.

Sonuç: test edilen beş grubun dördünde bilgi alt sınırı tek LUT seviyesinde ve K ≤ 6 ile, yani state biti başına tek bir LUT6 ile gerçekleşiyor. PSL(2,8)'de gerçekleşmiyor; en azından bu yapıyla. Genel bir gerçekleştirmenin (SAT) PSL(2,8)'de K = 6 ile 9–11 bite inip inemeyeceği açık. Belirleyici olan, grubun küçük indeksli alt grup ailelerinin kesişim yapısı gibi görünüyor.

5.4 Fiziksel karşılık

XC7A100T üzerinde, 120 bitlik state'e kopyalanarak ölçüldü (Yosys + nextpnr-himbaechel, iki seed):

Gerçekleştirme	State	Döngü	Fmax (MHz)
Öğrenilen mimari, tam tablolu 3-bit bloklar (8:1 mux)	24	2 seviye	482 / 490
Elle, ters kod r = p^{-1}, 5 koordinat × 3 bit	15	LUT5	651 / 724
β_6: V4 + A4 coset'leri	7	LUT6 / LUT5	638 / 589
SAT, 6 bit	**6**	LUT6	665 / 622

Ölçüm düzeltmesi (29 Ekim, ikinci denetimin "gerçek geri besleme yolunu raporlayın" isteği üzerine). İlk ölçümlerde state, LED'e indirgenirken 2–3 LUT derinliğinde bir XOR aşamasından geçiyordu. Kritik yol analizi gösterdi ki tek seviyeli bütün tasarımlarda sınırı döngü değil bu aşama koyuyordu: eski değerler (378–473 MHz) düzeneğin sınırıydı. Düzenekte artık her LUT'tan sonra

register var; döngü dışındaki her yol tek LUT. Yukarıdaki değerler yeni ölçüm, ve her sonuçla birlikte kritik yolun mantığı da kaydediliyor (`loop_*.json`, alan `path`).

Bundan çıkan sonuç: Daha kompakt encoding, ölçülen iki yerleştirmede belirgin bir Fmax bedeli ödetmedi. 6 bitlik mantıksal optimum ~640 MHz'e ulaştı; 7 ve 15 bitlik karşılıklarla aynı hız ölçeğinde (589–724). Aralıklar örtüşüyor ve iki seed bir sıralama vermiyor. S, L ve R birlikte karşılaştırılmadan 6 bitlik tasarımın fiziksel Pareto optimumu olduğu söylenemez. Tam tablolu öğrenilen mimari ise hem daha fazla bit hem daha uzun döngü kullanıyor: gereğinden esnek. **Transition codebook** fikrinin dayanağı bu.

Başka gruplarda fan-in ve Fmax (`loop_groups.json`, `loop_realize.json`, `loop_psl28.json`; ~120 bite kopyalanarak, iki seed ortalaması, düzeltilmiş düzenek; LUT ve MUXF sayıları 120 bitin toplamı; "kritik yol" = en yavaş yoldaki mantık, iki seed):

Grup	Kod	State	Fan-in	Fmax (MHz)	Kritik yol	LUT	MUXF7/8
A_5	dense	6	8	333	2 MUXF (+2 LUT)	607	334
	lemma (rastgele transversal)	6	7	417	3 LUT / LUT + MUXF	258	17
	gauge çözülmüş	6	6	**643**	1 LUT	145	0
	birleşik, K = 5	7	5	633	1 LUT	144	0
	positional (ters kod, base 3)	9	5	599	1 LUT	142	0
PSL(2,7)	dense	8	10	193	5–6 LUT	4 152	976
	lemma	8	9	288	4–5 LUT	1 052	306
	gauge	8	8	322	2–3 LUT (+MUXF)	706	322
	birleşik, K = 5	8	5	**669**	1 LUT	145	0
	positional	9	5	634	1 LUT	142	0
A_6	dense	9	11	156	4–6 LUT (+MUXF)	6 232	2 275
	lemma	9	10	232	4–5 LUT (+MUXF)	2 236	512
	gauge	9	9	280	3–4 LUT (+MUXF)	1 510	627
	birleşik, K = 6	9	6	**636**	1 LUT	142	0
	positional	12	5	642	1 LUT	145	0
PSL(2,8)	birleşik, K = 8 (dense core + Z_2^3)	9	8	317	2–4 LUT (+MUXF)	641	314

Grup	Kod	State	Fan-in	Fmax (MHz)	Kritik yol	LUT	MUXF7/8
	birleşik, K = 7	10	7	412	1–3 LUT (+MUXF)	245	57
	birleşik, K = 6 (positional)	12	6	**641**	1 LUT	145	0

Okuma:

- **Fan-in 6, fiziksel bir eşik.** K ≤ 6 olan bütün tasarımlar, kod ve state biti ne olursa olsun, kritik yolda tek LUT ile 599–669 MHz veriyor; aralarındaki fark seed gürültüsü düzeyinde. K > 6'da 6'dan fazla girişli fonksiyon LUT6'ya sığmıyor: MUXF7/F8 ya da ek LUT seviyeleri giriyor ve Fmax düşüyor.
- **Aynı state'te kazanç büyük.** A6, 9 bitte: fan-in 11 → 6 ile 156 → 636 MHz (4,1 kat) ve 6 232 → 142 LUT (44 kat). PSL(2,7), 8 bitte: 193 → 669 MHz (3,5 kat). A5, 6 bitte: 333 → 643 MHz (1,9 kat).
- **PSL(2,8)'de en az state'li tasarım en yavaşı.** 9 bit / fan-in 8: 317 MHz; 12 bit / fan-in 6: 641 MHz. Fiziksel optimum, LUT6 eşiğinin altında kalan en küçük state.
- **Geri çekilen yorum.** İlk ölçümde K ≤ 6 tasarımları arasında positional kod daha hızlı görünüyordu ve bunu yerelliğe bağlamıştım. Bu, eski düzeneğin çıkış ağacının bir artefaktıydı. Düzeltilmiş ölçümde K ≤ 6 içinde ayırt edilebilir bir sıralama yok.
- **Yani teorinin donanımdaki karşılığı:** önce fan-in ≤ 6 (LUT6 eşiği). Bu eşiğin altında, ölçtüğümüz boyutlarda state biti sayısı Fmax'i belirgin biçimde etkilemiyor; yalnız LUT ve register sayısını etkiliyor.

Kapsam. Bu benchmark yalnız state döngüsünü ölçüyor. Olasılık head'i, model parametreleri ve entropy coder dahil değil. ~640 MHz, tamamlanmış bir sıkıştırıcının byte/s değeri olarak okunmamalı.

Öğrenilen sınıfın içinde (`probe_a5embed.json`). Doğru çözüm, eğitilen modelle birebir aynı mimarinin içine yazıldı: 3-bit bloklu `fbseq.Gated`, gate MLP'leri ile. Gate ağlarına tam tablolar yerleştirildi; üç blok $r(j) = p^{-1}(j)$ değerlerini tutuyor (24 bitin 9'u). Sonuç:

- state'ten doğrudan çözülen grup elemanı: %100;
- yalnız head eğitildiğinde, 64 ve 256 uzunlukta: %100.

Yani bu sınıf A5'i temsil edebiliyor; eğitimde görülen başarısızlık bir öğrenme sorunu. Burada state tablolarının doğruluğu tam (yapı gereği); head'in başarısı ise yalnız test edilen uzunluklar için ölçüldü.

5.5 Açık sorular

1. K = 5 ve K = 4'te block-local olmayan en küçük state. A5, S = 6, K = 5: z3 her üç harf kodlamasında 6 saatte unknown (§5.3). Sorgu bit permütasyonu (6! = 720) ve sol çarpma (60) simetrilerini kırmıyor; bunları kıran bir CNF kodlaması ve CDCL çözücüsü sıradaki deneme.
2. Genel bir G için β_K ile block-local olmayan minimum arasındaki ilişki. Grup teorisinde yakın kavramlar var (minimal faithful degree, base size); tam olarak bu ölçü bulunamadı.

3. Cascade'lerde etkin alfabe (§3 uyarısı) ve injective olmayan encoding'ler (state splitting).
4. Encoding bulmanın karmaşıklığı. Sabit bir netlist'te en küçük döngü derinliği polinom zamanda bulunuyor (Pan–Liu); zorluk tamamen encoding seçiminde.
5. Derby'nin (2001) CRC/LFSR için state-space dönüşümüyle döngüyü kısaltmasının doğrusal olmayan genellemesi bulunamadı. Teorem C için doğal bir şablon.
6. **Düzeltildi: tersinir devre paritesi uygulanamaz.** Önceki sürüm, Shende vd. (2003) Lemma 11'den "her adımda en az bir biti değiştirmeyen devreler yalnız çift permütasyon üretir" sonucunu çıkarıyordu. Bu yanlış.
7. Karşı örnek: CNOT, F(a, b) = (a, b ⊕ a). a hiç değişmiyor, ama 10 ile 11 yer değiştiriyor; bu tek bir transpozisyon, yani tek permütasyon. Dört kodun hepsi kullanılıyor ve ek bit gerekmiyor.
8. Lemmanın gerçek koşulu: n hatlı tersinir devrede hiçbir kapı n hattın tamamına dokunmaz. Kapının dışında kalan hat kontrol olarak da okunmaz. Bir biti değiştirmeden kontrol olarak okumak bu koşulu sağlamaz.
9. Paralel LUT güncellemesinin bir tersinir kapı dizisine eşdeğer olduğu da ayrıca gösterilmeliydi.

Sonuç: tam 2^S kod ve K < S fan-in ile tek permütasyonlar için bir parite engeli olup olmadığı açık bir soru. 7. Coset core + fiber (§5.3): gauge her etiketlemede çözülmüyor; A5'te sertifikalı karşı örnek var, rank koşulu Düzeltme 2'de. Açık olan: hangi etiketlemeler çözülüyor, ve kontrol bitlerinin seçilmesiyle birleşen bir gauge ne veriyor? H abelian değilse fiber, H'nin bütün elemanlarını harf olarak alan bir alt makinedir (§3 uyarısı); bunun maliyeti? 8. Birleşik yapı: hangi gruplarda alt sınır K = 6 ile ulaşılabilir? PSL(2,8) bu yapıyla ulaşamıyor (§5.3). Küçük indeksli alt grupların kesişim yapısına dayanan bir koşul var mı? Genel gerçekleştirme (SAT) PSL(2,8)'de K = 6 ile 9–11 bite inebilir mi? Diğer açık durumlar: A_5'te K = 5'te 6 bit (SAT 6 saatte unknown), A_6'da K = 5'te 9–11 bit, ve bütün gruplarda K = 4.

---

6. İfade gücü sınıfları

Ölçtüğümüz Boolean sıralama (64'te eğitim, 256'da test, 3 seed):

Model	flag	parity	mod 3	A5
set/keep	1,00	0,51	0,34	0,03
+ toggle	1,00	**1,00**	0,34	0,03
2-bit blok	1,00	1,00	**1,00**	0,05
3-bit blok	1,00	1,00	1,00	**0,32**

A5 sütunundaki 0,32, üç seed'in ortalaması: 0,12, 0,83 ve 0,02.

Bilinen karşılıklar:

- Sarrof, Veitsman, Hahn (NeurIPS 2024): nonnegative SSM ⇔ star-free.
- Grazzi vd. (ICLR 2025): parity için negatif eigenvalue; 2'nin kuvveti olmayan m ile mod m için non-real eigenvalue.
- Krohn–Rhodes, Schützenberger, McNaughton–Papert: star-free ⇔ aperiodic.
- Barrington (1989): non-solvable grupların word problem'i NC^1-complete.
- Knorozova & Ronca (AAAI 2024): cascade çerçevesi.

Sıralamanın sürekli halinin teorisi yani zaten var; bizim katkımız Boolean blok karşılığı ve ölçümü.

Sanı (literatür ajanının Krohn–Rhodes'tan türettiği bir taslak, bizim tarafımızdan kontrol edilmeli): Girdi kontrollü k-bitlik bloklarla tanınabilen düzenli diller tam olarak şunlardır: sözdizimsel monoidin her basit grup böleni S_{2^k}'yi böler.

Test edilebilir öngörüleri:

- mod 5 ve mod 7, 2-bit bloklarla hiçbir derinlikte çözülemez, tek bir 3-bit blokla çözülür;
- mod 6 ve S_4, 2-bit bloklarla çözülür;
- Z_{11} 4-bit blok ister;
- Z_{2^j}, 1-bit bloklarla j katman ister.

---

7. Öğrenilebilirlik

İki ayrı ölçü (denetimden sonra ayrıldı; önceki sürüm ikisini karıştırıyordu):

- **Aynı parametrede discretization farkı:** Δ_disc(θ) = L_hard(θ) − L_soft(θ), aynı veride.
- **Performans açığı:** hard modelin kaybı ile ayrı eğitilmiş bir soft referansın kaybı arasındaki fark. Farklı parametre noktalarını karşılaştırır.

Ölçümler (Loghub, 256 B paket, set/keep, `gap.json`):

Parametre noktası	Soft bpb	Hard bpb	Δ_disc = hard − soft
Soft eğitimden sonra	1,3976	2,7693	+1,3717
Hard'a uyarlamadan sonra	1,8168	1,5829	−0,2339
Baştan hard-forward, eğim tavlaması (`probe_*.json`, yeni val yarısı)	2,7501	1,2560	−1,4941

Aynı parametrede fark işaret değiştiriyor. Hangi forward'la eğitilirse model o forward'a uyuyor, diğerinde bozuluyor.

Önceki sürümdeki "Δ_hard = 0,185" ise 1,5829 − 1,3976'ydı; yani uyarlanmış hard modelin soft referansa göre **performans açığı**. Soft model state'leri sürekli kullanıyor: ortalama min(s, 1−s) 0,16–0,21. Yani soft model ile Boolean makine aynı fonksiyon sınıfında değil.

Hard-forward eğitim (Loghub, set/keep, 2 seed):

- Baştan Boolean forward, backward'da sigmoid(a·l), a 1'den 5'e (eğim tavlaması): **1,418 bpb** (1,4186 ve 1,4176).
- Pencere 8 modelini (1,453) geçiyor.
- Eski soft referansa (1,398) açığı 0,02.
- Eğitim ve değerlendirme aynı deterministik hard forward'u kullandığı için seçim uyumsuzluğu zaten sıfır. 0,02, eski soft optimuma eşit kalite garantisi değildir.
- Kapsam: iki seed; test aynı dosyaların son %20'si; yöntem bu validation üzerinde geliştirildi. Görülmemiş sistemlerde üstünlük henüz gösterilmedi.
- Sade Hard-ST (a = 1): 1,545.

- Gumbel-ST (τ 1 → 0,25): 1,685 (1,6905 ve 1,6799).
- Kaynak: ilk koşu, `model/fbseq/archive/hardaware_bytes_run1.json`. O koşudaki eğim hatası yalnız Hard-ST'yi etkiliyordu; Gumbel'in surrogate ölçeği 1/τ olduğu için etkilenmedi.
- Düzeltilmiş kodla yeniden üretildi: 1,6905 ve 1,6799, birebir aynı (`hardaware_bytes.json`).
- Kim'in (2026, arXiv 2603.14157) düşük sıcaklıkta Gumbel-ST çöküşü bulgusuyla uyumlu.
- Toggle'lı model: eğim tavlamasıyla 1,462.
- 2-bit ve 3-bit bloklar, eğim tavlamasıyla (1 seed): **1,680** ve **2,237**.
- Boolean sıralama Loghub'da ifade gücünün tersine dönüyor: set/keep 1,418 < toggle 1,462 < 2-bit 1,680 < 3-bit 2,237.
- İki karışık etken var. Eşit (~100K) parametre bütçesinde blok büyüdükçe gate ağı daralıyor (gizli genişlik 80 → 48 → 26). Ayrıca gate ağının çıkışı büyüyor: state biti başına 2, 4 ve 8 tablo logit'i (2^kb durumun her biri için bir değer).
- Yani "log verisi grup benzeri durum izlemeye ihtiyaç duymuyor" sonucu henüz çıkarılamaz. Ayırmak için gate genişliği sabit tutulmalı ya da set/keep katmanı ile küçük bir blok katmanı birleştirilmeli.

Not (Gumbel). 1[(l+n)/τ > 0] = 1[l+n > 0]. τ'yu düşürmek forward'daki gürültüyü azaltmaz, yalnız backward surrogate'ini değiştirir. Uygulamamız standart Gumbel-ST: logistic gürültü, yani iki Gumbel'in farkı; forward'da sabit gürültü. Gürültünün kendisini sönümleyen bir varyant denenmedi.

A5 trainability (3-bit blok, 4 seed, kararlı relaxation, `hardaware_a5.json`; 256 adımda doğruluk):

softhard seed	En iyi	Son
0	1,0000	1,0000
1	0,3612	0,2065
2	1,0000	1,0000
3	0,2476	0,0917

- Elle kurulan devre %100 (representability; tam geçiş denetimi).
- soft→hard: 4 koşunun 2'si, test edilen 64 ve 256 uzunlukta %100. Bu, bütün uzunluklar için makine eşdeğerliğinin kanıtı değil.
- Başarısız iki koşu farklı davranıyor: biri en iyi 0,36'dan 0,21'e, diğeri 0,25'ten 0,09'a geriliyor.
- **Coset aşaması yalnız bir hipotez.** İç temsil Z_3'ün 20 coset'ini ayırıp coset içindeki 3 elemanı ayıramıyorsa, doğruluk 1/3 olur (Zhang & Li 2026). Ama bu ileri yönlü bir çıkarım; doğruluk sayısından temsile geri gidilemez. Bunu sınamak için iç durum kümelerine, SP özelliğine ya da stabilizer'a bakmak gerekir.
- Baştan hard (Hard-ST ve eğim tavlaması): 8 koşunun hiçbiri 0,13'ü geçmiyor.
- **Ödünleşim, denenen reçetelerle sınırlı:** dil modelinde baştan Boolean eğitim en iyisi. A5'te softhard 4 koşunun 2'sinde başarılı, iki hard-forward yöntemin 8 koşusunun hiçbirinde değil. Başka reçetelerin, sürelerin ya da başlangıçların başarısız olacağı gösterilmedi; "soft aşama şart" denemez.

Transition codebook: olumsuz sonuç (`codebook_*.json`). Her 3-bitlik blok, m = 8 öğrenilmiş dönüşümden birini seçiyor. A5'te 256 adımda, en iyi değerler:

Başlangıç	softhard (4 seed)	Eğim tavlaması (4 seed)
Rastgele	< 0,06	< 0,06
Identity'ye yakın	0,025–0,032	0,028–0,029
Identity'ye yakın, tablo logit'leri × 10	0,108–0,117 (son 0,04–0,11)	0,038–0,046
Doğru dönüşümler, donmuş (yalnız seçim ağı ve head öğreniyor)	0,11–0,36 (son 0,02–0,09)	0,02–0,10

Loghub'da rastgele başlangıçla 2,07–2,16 bpb.

Teori, A5'in tek seviyede yalnız 3 dönüşümle temsil edilebildiğini söylüyor. Ama dönüşümleri ve seçimi birlikte öğrenmek denenen 24 koşunun hiçbirinde çalışmadı. En iyi değer 0,12; şans düzeyinin (1/60 ≈ 0,017) üstünde, %100'ün çok altında. Bu haliyle codebook bir öğrenme yöntemi olarak başarısız. Bir donanım hedefi olarak geçerliliği (§5.4) bundan etkilenmiyor.

Ayrıştırma: doğru dönüşümleri bulmak mı, seçmek mi? (ikinci denetimin önerisi; `codebook.py --init fixed`, `--curriculum`; `hardaware.py --curriculum`)

Kurulum: layer 0'ın 0–2 numaralı blokları, ters kod $r(n) = p^{-1}(n) \oplus n$'yi tutuyor. Codebook'un ilk üç girdisi üç harfin doğru dönüşümleri. Tablolar donmuş; yalnız seçim ağı ve head öğreniyor. Doğru seçimle model grubu 2048 / 2048 adımda izliyor, yani doğru çözümü ifade edebiliyor. Curriculum: eğitim uzunluğu, adımların ilk 2/3'ünde 4'ten 64'e çıkıyor.

Dönüşümler	Codebook	Eğitim uzunluğu	softhard	slope
Doğru, donmuş	m = 3, yalnız doğrular	64	**4/4**	0/4 (≤ 0,10)
Doğru, donmuş	m = 8, ek girdiler identity	64	0/4 (en iyi 0,36)	0/4 (≤ 0,10)
Doğru, donmuş	m = 8, ek girdiler rastgele permütasyon	64	0/4 (≤ 0,13)	0/4 (≤ 0,10)
Doğru, donmuş	m = 8, ek girdiler identity	4 → 64	**3/4**	**1/4**
Öğreniliyor	B_block3 (tam tablo)	64	2/4	0/4 (Hard-ST 0/4)
Öğreniliyor	B_block3 (tam tablo)	4 → 64	0/4 (en iyi 0,32)	1/4
Öğreniliyor	codebook m = 8, identity'ye yakın başlangıç	64	0/4	0/4

Dönüşümler	Codebook	Eğitim uzunluğu	softhard	slope
Öğreniliyor	codebook m = 8, identity'ye yakın başlangıç	4 → 64	0/4 (≤ 0,03)	0/4 (≤ 0,03)

"k/4": dört seed'in kaçı test edilen 64 ve 256 uzunlukta yuvarlanmış %100'e ulaştı. Doğruluk dört basamağa yuvarlanıyordu; 256 uzunlukta 1,0000, 65 536 tahminde en fazla 3 hata demek. Kesin sıfır hata bu kayıtlardan gösterilemiyor. Yeni koşularda ham hata sayısı kaydediliyor (`last_wrong`, `min_wrong`). Tablo 48 koşudan oluşuyor: 40 codebook, 8 B_block3.

Okuma (üçüncü denetimden sonra daraltıldı):

- **Eşleşen karşılaştırma.**
- Curriculum yalnız "identity kopyalı" codebook'la denendi: softhard 0/4 → 3/4, slope 0/4 → 1/4.
- Rastgele permütasyon girdili codebook'la curriculum henüz denenmedi. Önceki sürüm iki ayrı koşulu "0/8 → 4/8" diye havuzluyordu.
- **Identity kopyaları "yanlış dönüşüm" değil.** Eşit logit'lerde identity'ye düşen ağırlığı 1/3'ten 6/8'e çıkarıyorlar. Yani müdahale yalnız seçenek sayısını değil, başlangıçtaki işlem tercihini de değiştiriyor. Rastgele permütasyonlar ise kullanılmayan kodlara da gidiyor.
- **Curriculum'un etkisi var, mekanizması izole değil.**
- Daha fazla eğitimle açıklanamıyor: curriculum, sabit uzunluğun %56'sı kadar sembol işliyor (6,86 M'ye karşı 12,29 M).
- Ama head de birlikte öğreniliyor ve kısa dizilerde hedef dağılımı değişiyor. "Uzun ufuklu kredi ataması" bu yüzden bir hipotez; gösterilmiş değil.
- **Identity başlangıcında seçime giden gradyan tam sıfır olabilir.** $M = \Sigma\pi_i M_i$ için $\partial M/\partial z_j = \pi_j(M_j - M)$. Bütün prototipler aynı identity ise bu sıfır. Identity'ye yakın başlangıçta hard tabloların hepsi büyük olasılıkla identity'ye yuvarlanıyor (işaret değişimi olasılığı $\leq 3\cdot10^{-6}$). Yani seçim ağı, tablolar farklılaşana kadar bu yoldan öğrenemiyor. Bu, "dönüşümleri bulmak özünde zor" yorumuna karşı test edilebilir bir alternatif.
- **Donmuş tablolar soft aşamada tam Boolean değildi.** Logit'ler ±6, yani yanlış bit olasılığı 0,0025; 64 adımda bu birikebiliyor. Saf seçim deneyi için tablolar tam 0/1 tutulup yalnız seçim gevşetilmeli (`--exact`).
- **Eksik köşe: oracle seçim + öğrenilen tablolar.** Seçim harfe sabitlenip yalnız tablolar ve head öğrenilirse tablo keşfi seçimden ayrılır (`--oracle`).
- **Takılan koşular sık sık ~1/3'te duruyor** (0,32–0,40). Bu, coset hipoteziyle uyumlu, ama doğruluk sayısından temsil çıkarılamaz.
- **Savunulabilir sonuç.** Teoriden gelen sabit dönüşüm aileleri, denenen kurulumlarda tablo öğreniminden açıkça daha iyi bir başlangıç veriyor. Seçim öğrenimi ise reçeteye ve aday ailesine duyarlı. Tabloların öğrenilememesi, seçimden bağımsız bir sorun (oracle 0/24, üç başlangıçla).

Tamamlayıcı deneyler (`learn2/*.log` → `codebook_a5.json`; ham hata sayılarıyla; "sıfır hata" = son modelde 64 ve 256 uzunlukta 0 / 16 384 ve 0 / 65 536 hata):

Kurulum	softhard	slope
Oracle seçim + öğrenilen tablolar, identity'ye yakın başlangıç	0/4 (≈0,03)	0/4 (≈0,03)
Oracle seçim + öğrenilen tablolar, rastgele başlangıç	0/4 (≈0,03)	0/4 (≈0,04)
Doğru tablolar tam 0/1, m = 3	**4/4 sıfır hata**	–
Doğru tablolar tam 0/1, + identity kopyaları	0/4 (en iyi 0,35)	–
Aynısı, curriculum	**2/4 sıfır hata** (biri 4 151 hata, biri 0,34)	–
Doğru tablolar tam 0/1, + rastgele permütasyonlar	0/4	–
Aynısı, curriculum	0/4	–
Doğru tablolar (±6 logit), + rastgele permütasyonlar, curriculum	0/4	0/4
Öğrenilen codebook, rastgele başlangıç, curriculum	0/4	0/4
Oracle seçim + öğrenilen tablolar, küçük logit'li başlangıç (≈0, doymamış)	0/4 (≈0,04)	0/4 (biri 0,54'e tırmanıyor, diğerleri ≤ 0,13)
Öğrenilen codebook, küçük logit'li başlangıç	0/4	0/4 (≤ 0,13)

Slope zaten baştan hard tablo kullandığı için "tam 0/1" yalnız softhard'ı değiştiriyor.

Okuma:

- **Oracle köşesi: seçim verilse bile tablolar öğrenilemiyor** (0/24, üç başlangıçla; çoğu şans düzeyine yakın). Yani codebook'taki başarısızlık, seçim ile tabloların birlikte öğrenilmesindeki bir koordinasyon sorunu ya da seçim gradyanının identity simetrisinde sıfırlanması değil. Kaynak, tablo parametrelemesi / başlangıç / yörünge üzerinden öğrenme.
- **Seçim kısmı aday ailesine duyarlı.**
- Yalnız doğru dönüşümler: kolay (4/4, sıfır hata).
- Identity kopyalarıyla: curriculum gerekiyor (2/4 ve 3/4).
- Rastgele permütasyon çeldiricileriyle: curriculum dahil 0/12.

Curriculum'un yararı yalnız identity kopyalı ailede görüldü. - **Tam 0/1 tablolar** (soft aşamada yumuşama yok) sonucu belirgin biçimde değiştirmedi: identity kopyalılarda 0/4 → 0/4, curriculum'la 3/4 → 2/4. - **Codebook ile B_block3 arasındaki fark.** Oracle'lı codebook, işlevsel olarak "harfe göre tablo seçen" bir B_block3'e denk; ama B_block3 bazen öğreniyor (2/4), codebook hiç öğrenmiyor. Fark parametrelemede. Codebook tabloları ±3 logit'le, yani doymuş sigmoid'le başlıyor (türev ~0,045). B_block3'ün tabloları ise gate ağından ~0 logit'le, yani maksimum belirsizlikle başlıyor.

Küçük logit'li başlangıç bunu ayırdı: doygunluk tek başına açıklama değil. Oracle'la bir slope koşusu 0,54'e kadar basamak basamak tırmanıyor, ama hiçbir koşu başarıya ulaşmıyor. Kalan fark şu: codebook'ta tablolar doğrudan parametre, B_block3'te ise paylaşılan bir gate ağından üretiliyor (aşırı parametreleme). Bu bir hipotez.

Literatürdeki reçeteler:

- PD-SSM (Terzić vd., NeurIPS 2025): one-hot geçiş matrisleri, forward hardmax, backward softmax; A5 tek katman %100.
- Mind the Gap (NeurIPS 2025, feedforward): Gumbel + ST.
- Kim 2026: Hard-ST ve CAGE.
- HM-RNN (ICLR 2017): hard ST ve eğim tavlaması; hard model soft modeli geçmiş.
- R-DTLGN (2026): recurrent modellerde hatanın döngüde büyümesi, trajectory distillation.

---

8. Kararlılık

Hata rekürsiyonu (set/keep, soft ve hard karşılaştırması). $e_t = |s_t - \hat{s}_t|$ için:

$e_t \le (1-g_t)\cdot e_{t-1} + g_t\cdot|u_t - \hat{u}_t| + |g_t - \hat{g}_t|$

Toggle'lı model ($s_t = b_t + c_t\cdot s_{t-1}$, $c_t = a_t - b_t \in [-1, 1]$) için:

$e_t \le |c_t|\cdot e_{t-1} + |b_t - \hat{b}_t| + |c_t - \hat{c}_t|$

Yörünge boyunca açık sınır. $a_t = 1 - g_t$ ve $\varepsilon_t = g_t|u_t - \hat{u}_t| + |g_t - \hat{g}_t|$ yazılırsa:

$e_t \le e_0 \prod_j a_j + \Sigma_k \varepsilon_k \prod_{j>k} a_j$

Bu, ortalama almadan her yörünge için geçerlidir. Her adımda $a_t \le \rho < 1$ ve $\varepsilon_t \le \varepsilon$ ise $e_t \le \rho^t e_0 + \varepsilon(1-\rho^t)/(1-\rho)$; ayrıca $e \le 1$.

Önerme (input-gated yapıda hata büyümez). Gate'ler state'e bağlıysa, $|g(s) - \hat{g}(h)| \le L_g\|s-h\| + \delta$ gibi bir bağıntı gerekir. Toplam katsayı 1'i aşabilir ve daralma kaybolur; GRU gibi hücrelerde durum budur. B'de ise gate'ler katmanın **kendi** state'ine bağlı değildir:

- Bir katmanın içinde ε_t yalnız girdiye ve alt katmanların state'ine bağlıdır; kendi hatasından bağımsızdır. Yukarıdaki sınır ek varsayım gerektirmeden geçerlidir.
- Katmanlar arasında hata ileri yönde akar: $e^{(l)}_t \le \rho\cdot e^{(l)}_{t-1} + L\cdot e^{(l-1)}_t + \delta$. Alt katman sınırlıysa üst katman da sınırlıdır.

Sonuç: **kb = 1'de state döngüsü discretization hatasını çarparak büyütmez** (çarpan $a_t \le 1$). Aynı yapı paralel scan'i ve tek mux'luk döngüyü de sağlıyordu.

Kapsam (ikinci denetimden):

- **Dış terimler birikir.** Her adımda gelen ε_t terimleri toplanır. $\rho < 1$ ise sınır $\varepsilon/(1-\rho)$'da doyar. $\rho = 1$ ise geometrik bir plato yoktur; sınır $t\cdot\varepsilon$'a kadar büyüyebilir.
- **kb > 1 bloklarda Hamming uzaklığı büyüyebilir.** Tablo input-gated ve state'ten bağımsız seçilse bile, blok içindeki map bitleri karıştırır. Örnek: F(x, y, z) = (x, x, x), 000 ile 100 arasındaki 1 bitlik farkı 000 ile 111 arasındaki 3 bitlik farka çevirir.
- **Dağılım relaxation'ı başka bir metrikte daralır.** Aynı sütun-stokastik M altında L1 (total variation) uzaklığı büyümez. Bu, Hamming uzaklığından farklı bir ölçü ve farklı bir varsayım.

Sızdıran bellek: temsil farkının olası bir kaynağı. Soft model $s_t = r \cdot s_{t-1}$ ile ($g = 1 - r$, $u = 0$) yavaşça unutabilir; Boolean bit ya tam tutar ya sıfırlar. Bu durumda tam çözüm $e_t = 1 - r^t$ olur. $r = 0{,}999$ için $e_{256} \approx 0{,}23$, $e_{4096} \approx 0{,}98$. Yani adım başına çok küçük bir gate farkı, uzun bir akışta büyük bir fark yaratabilir; bu örnekte sınır tam olarak eşitliktir. Soft state'lerin 0 ve 1'den uzak olması (0,16–0,21) böyle kısmi unutmanın izi olabilir. Boolean dünyada sızdıran belleğin karşılığı **sayaçlardır**: "k adım sonra unut" diyen çok bitli bloklar. Bu bir hipotez.

Koşullu sınır. $E[|c_t| \mid \mathcal{F}_{t-1}] \le \rho < 1$ ve adım başına ek hata $\le \varepsilon$ ise:

$$E[e_t] \le \rho^t \cdot E[e_0] + \varepsilon(1 - \rho^t)/(1 - \rho)$$

$\mathbb{F}_2$ tam eşitliği (hard, tek bit). Dört map'in hepsi $\mathbb{F}_2$ üzerinde affine: $s_{t+1} = a_t s_t \oplus b_t$. Aynı kontrol dizisi altında başlangıç farkı için:

$$s_t \oplus s'_t = \left(\prod_i a_i\right)(s_0 \oplus s'_0)$$

Fark ilk reset'e kadar korunur, reset'te silinir.

Bloklara genelleme. Aynı kontrol dizisiyle ilerleyen iki yörünge, iki durumu aynı duruma eşleyen bir adımda kalıcı olarak birleşir. Permütasyon adımları iki farklı durumu birleştiremez; durumları ayrı tutar, ama Hamming uzaklığını korumak zorunda değildir. Krohn–Rhodes diliyle:

- reset bileşenleri farkı siler;
- grup bileşenleri farkı taşır.

Koşul önemli: "aynı sonraki kontroller". Çok katmanlı devrede alt katmandaki bir hata üst katmanın kontrol dizisini değiştirebilir. Bütün devre için kontrol farkları da hesaba katılmalı.

Bellek ile dayanıklılık: yalnız bağımsız 1-bit keep/toggle için bağlı. Önceki sürüm "uzun hatırlayan bitler hataları da uzun süre taşır" diyordu. Bu, bağımsız tek bitlik keep/toggle için doğru: $\mathbb{F}_2$ eşitliği gereği fark ilk reset'e kadar yaşar. Genel Boolean bloklarda yanlış.

- **Karşı örnek (majority).** Mantıksal 0 = 000, mantıksal 1 = 111, F(s) = (maj(s), maj(s), maj(s)). Geçerli iki kod sonsuza kadar korunur. Herhangi bir bit çevrilirse bir sonraki adımda doğru kod geri gelir. Her çıkış bir LUT3. Sekiz durumun hepsi kontrol edildi.
- **A5'e bağlanan sınır.** 60 mantıksal durumu her tek bit hatasından düzelten bir d-bit kodun minimum uzaklığı en az 3 olmalı. Yarıçapı 1 olan Hamming topları ayrık olacağından $60 \cdot (d+1) \le 2^d$ gerekir. d = 9'da 600 > 512, yani $d \ge 10$.
- Kısaltılmış [10, 6, 3] Hamming kodu 64 kelime ve minimum uzaklık 3 ile bunu sağlıyor; 60 kelimelik bir alt kümesi yeter.
- Yani LUT fan-in'i ve gecikme kısıtı yokken **10 bit tam minimum** (`witness_check.json`).
- **Hamming sınırı 6 bitlik optimumla çelişmez.** Amaçlar farklı: 6 bit tam temsil için, 10 bit tek hata düzelten state kodu için optimal. Bu sınır, K = 6 ile tek seviyede hata düzelten bir geçiş devresinin var olduğunu göstermez. Bu açık bir soru.

Kayba bağlantı. Cross-entropy'nin logit gradyanının ℓ_1 normu en fazla 2. Head'in state'e göre $\ell\infty$ Lipschitz sabiti L ise:

$$|\Delta \mathrm{BPB}| \le (2L / (T \ln 2)) \cdot \Sigma_t \, \| s_t - \hat{s}_t \|_\infty$$

Ölçüm. D(t), soft ve hard state'lerin farklı olan bit oranı: birinci katmanda ~%6, ikinci katmanda ~%11–13; t ≈ 8–16'da doyuyor (`gap.json`).

Bu bölümün matematiği standart araçlardan türetiliyor. Bizim kısmımız, ölçümler ve bunların donanımdaki döngü yapısıyla ilişkisi.

Düzeltme: D(t)'nin doyması bellek kalitesini kanıtlamaz. Eşik uyuşmazlığı D_t ile sürekli uzaklık E_t farklı şeyler ölçer: $E_t \le Q_t + D_t$ ve $D_t \le 2E_t$, burada Q_t yuvarlama uzaklığı. Sabit bir uyuşmazlık oranı, eski bilginin hızla silinmesiyle de oluşabilir. Örnek: her adımda r olasılıkla iki yörünge aynı değere sıfırlansın, aksi halde p olasılıkla ayrışsınlar. Denge (1−r)p/(r+p−rp) olur; r = 0,15 ve p = 0,02 bizim eğrimize benzer bir doygunluk verir, ama başlangıç bilgisinin 256 adım hayatta kalma olasılığı $\sim 10^{-18}$'dir. Bu yüzden D(t)'nin doyması yalnız "uyuşmazlık incelenen ufukta artmıyor" demektir.

Uyarı (Lipschitz sınırı). Head'in ölçeği kontrol edilmezse sınır işe yaramaz. Tek bit için z = (M(2s−1), −M(2s−1)) alınırsa bir bitlik fark kayıpta 2M/ln 2 bit değişime yol açar. L, eğitilmiş head için ölçülmelidir.

Deney (`probes.py` → `probe_B_setkeep_slope_s0.json`): Hard modelde tek bir state biti çevrilir. Farkın yaşam süresi, üst katmana yayılması ve bpb'ye etkisi ölçülür. Ayrıca bir lineer probe ile state'in k adım önceki byte'ı ne kadar taşıdığına bakılır (gecikmeli okuma).

Model: set/keep, eğim tavlaması, seed 0. Değerlendirme yeni val yarısında, eski val'in ilk yarısı (1,256 bpb). İkinci yarı (test) 1,576 bpb veriyor; log dosyalarının sonu daha zor, yani aynı dosyanın içinde bile dağılım kayıyor.

Tek bit çevirme (birinci katman, $t_0 = 32$, 24 bit × 128 paket):

Ölçü	Değer
Farkın yaşam süresi, medyan / ortalama	5 / 23 adım
≥ 16 / ≥ 64 / paket sonuna kadar (224) yaşayan	%28 / %9,6 / %4,0
İkinci katmanda bozulan bit, adım başına ortalama (120 bitten)	0,51
Paketin geri kalanında Δbpb, ortalama / en kötü	+0,008 / +0,135

Bu, input-gated önermesiyle uyumlu: tek bir hata büyümüyor, üst katmana yarım bit kadar sızıyor ve kayba etkisi küçük. Kuyruk ise bellek–dayanıklılık bağını gösteriyor: çevrilen bitlerin %4'ü hatayı paket sonuna kadar taşıyor. Bunlar uzun tutan keep bitleri; bağımsız 1-bit keep/toggle için beklenen davranış bu.

Gecikmeli okuma (lineer probe; hedef k adım önceki byte). Kontrol: aynı probe yalnız pencereden (son w = 4 byte). Bu kontrol, log tekrarları yüzünden eski byte'ın yakın bağlamdan zaten ne kadar tahmin edilebildiğini ölçüyor. İlk sürümde bu kontrol yoktu.

k	1	4	8	16	32	64	128
State	0,58	0,53	0,31	0,29	0,25	0,20	0,18
Yalnız pencere	0,97	0,96	0,21	0,18	0,14	0,11	0,105
Çoğunluk sınıfı	0,11	0,11	0,11	0,11	0,11	0,11	0,11

k ≤ 4'te hedef pencerenin içinde, bu yüzden kontrol neredeyse tam. k ≥ 8'de state kontrolü her k'da geçiyor; k = 128'de kontrol çoğunluk düzeyine inmişken state 0,18'de kalıyor. Yani Boolean state,

pencerenin ötesindeki geçmiş hakkında, yakın bağlamın açıklayamadığı bilgi taşıyor. Bu bilginin harfiyen saklanan byte mı, yoksa satır şablonu gibi bir özet mi olduğunu probe ayıramaz.

Head'i yeniden eğitme (sonuçsuz). Gate'ler donduruldu ve head sıfırdan, eğitimdeki reçeteyle 3 000 adım eğitildi. Sonuç 1,461 bpb; orijinal head 1,256. İlk sürüm farklı bir reçete kullanıyordu ve 1,494 vermişti. Fark, state'in bilgi taşımadığı biçiminde okunamaz, çünkü özellikler aynı. Daha olası açıklamalar: 3 000 adım sıfırdan bir head için yetmiyor, ya da ortak eğitimin bulduğu çözüme ayrı eğitimle ulaşılamıyor. Daha uzun eğitimle tekrar edilmeli.

---

9. Literatürdeki konum

Bilinen ve kullandığımız:

- retiming ve iteration bound: Leiserson–Saxe 1991; Renfors–Neuvo 1981; Parhi–Messerschmitt 1991;
- FSM look-ahead: Lin–Messerschmitt 1991;
- partition pair ve reduced dependence: Stearns–Hartmanis 1961, kitap 1966;
- Krohn–Rhodes, holonomy;
- LUT tabanlı FSM synthesis: Barkalov vd. 2021;
- ifade gücü: Sarrof 2024, Grazzi 2025, Merrill 2024, PD-SSM 2025, Liu 2023, Barrington 1989;
- discretization: Mind the Gap 2025, Kim 2026, HM-RNN 2017.

En yakın rakipler:

- RDDLGN (EdgeFM 2025 atölyesi): concatenation recurrence, keep yolu yok, donanım yok; future work olarak associative recurrent blokları öneriyor.
- Ovin & Liwicki (NeurIPS 2026 AXIOM atölyesi): bit başına keep/write mux, sentetik recall; donanım yok. Tam metne erişilemedi.

Bulunamayanlar (bizim için boşluklar):

- sentezlenip Fmax'i raporlanmış bir recurrent logic ya da LUT modeli;
- logic ya da LUT ağlarıyla byte düzeyinde bpb ve kayıpsız sıkıştırma;
- döngüdeki LUT seviyesini tasarım kısıtı olarak kullanan bir çalışma;
- Boolean transition codebook;
- adım bazında discretization gap ölçümü;
- Boolean mux monoidinin scan ile eğitimi;
- A5 gibi gruplar için LUT fan-in kısıtlı en küçük gerçekleştirme sınırları (§5).

Ayrıntılı künyeler: `docs/08_case_arastirmasi.md`, `paper/related_work_2026.md` ve Faz 0 belgesinin kaynakçası.

---

10. Sonraki adımlar

1. Birleşik yapının genel bir koşulu: hangi G için bilgi alt sınırı K = 6 ile ulaşılabilir (§5.5, madde 8)? A5'te K = 5'te 6 bit: z3 6 saatte unknown; simetri kırmalı CNF ile yeniden denemek. Daha büyük gruplar (PSL(2,8), A7).

2. Positional ve birleşik tasarımların Fmax farkının daha çok seed ve yerleşim kısıtıyla ölçülmesi; yerelliğin bir maliyet terimi olarak modele eklenmesi.
3. S_4, mod 5 ve mod 7 ile §6'daki sanının testi.
4. A5'i güvenilir öğrenmek. Ayrıştırmayı tamamlamak (oracle seçim, tam 0/1 tablolar, eşleşen curriculum koşulları). Sıradakiler: dönüşümleri yapıcı inşadan alıp yalnız seçim ve head'i öğrenmek (Loghub'da da); PD-SSM reçetesi, CAGE; takılan koşularda iç durumun coset yapısını incelemek.
5. Hard-aware finalistlerin daha fazla seed'le tekrarı ve leave-one-system-out değerlendirme.
6. Kararlılık: L sabitinin ölçülmesi; K = 6 ile tek seviyede hata düzelten bir A5 devresi var mı?
7. Gate ağları ve head hâlâ float. LUT ağlarına çevrildiklerinde kalite ve döngü dışı mantığın maliyeti (Faz 1).